Oracle C2M Monitoring & Alerting with Dynatrace

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Monitoring & Alerting with Dynatrace 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Oracle C2M Monitoring & Alerting with Dynatrace 5

What this monitors . . . 5

Why this matters . . . 5

What Dynatrace gives you on this stack . . . 6

The four things to monitor across C2M / WebLogic . . . 6

Named objects this solution defines . . . 6

What this is not . . . 7

The deployment model . . . 7

2. Requirements Guide — Oracle C2M Monitoring & Alerting with Dynatrace 7

2.1 A Dynatrace tenant reachable from the C2M hosts . . . 8

2.2 A PaaS/installer token and an API token (least privilege) . . . 8

2.3 The C2M / WebLogic topology in scope . . . 8

2.4 OneAgent deployable on every C2M host (root/sudo, host group tag) . . . 8

2.5 WebLogic connection-pool / JMX metrics enabled . . . 8

2.6 Environment access to build dashboards, alerts, and notifications . . . 9

2.7 Alert destination and on-call routing decided . . . 9

2.8 A non-production pilot environment and change control . . . 9

Checklist . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 11

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

3. Implementation Guide — Oracle C2M Monitoring & Alerting with Dynatrace 12

Step 1 — Assess current coverage . . . 12

Step 2 — Instrument the tiers with OneAgent . . . 12

Step 3 — Define the management zone and environment tag . . . 13

Step 4 — Build the C2M dashboards . . . 13

Step 5 — Define the standardized alerts (custom metric events) . . . 13
Step 6 — Route alerts with an alerting profile and notification . . . 14
Step 7 — Pilot on non-production . . . 14
Step 8 — Validate against the pilot gate . . . 15
Step 9 — Expand to production . . . 15
5. Testing Guide — Oracle C2M Monitoring & Alerting with Dynatrace 15
Test 1 — Every in-scope tier is instrumented . . . 15
Test 2 — The environment tag is set . . . 16
Test 3 — Connection-pool metrics are present . . . 16
Test 4 — The dashboards render live C2M data . . . 16
Test 5 — The standardized alert rules exist and are scoped . . . 16
Test 6 — An alert fires on a real condition (non-production) . . . 16
Test 7 — The alert routes to the destination . . . 16
Test 8 — Coverage report is clean . . . 16
Pass/fail gate . . . 17
6. Troubleshooting Guide — Oracle C2M Monitoring & Alerting with Dynatrace 17
A. A host is not reporting an OneAgent . . . 17
B. A C2M JVM is instrumented but a metric is missing . . . 18
C. The environment tag is wrong or missing . . . 18
D. A dashboard tile is empty . . . 18
E. An alert does not fire . . . 18
F. An alert fires but nobody is notified . . . 19
G. Too many alerts / alert fatigue . . . 19
H. Diagnostics quick reference . . . 19
I. When to escalate . . . 19
7. Best Practices — Oracle C2M Monitoring & Alerting with Dynatrace 20
8. Deliverables Summary — Oracle C2M Monitoring & Alerting with Dynatrace 21
Documentation . . . 21
Named objects . . . 21
Automation scripts (shell & Dynatrace API via curl — not PowerShell) . . . 22
What the buyer receives . . . 22
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Oracle C2M Monitoring & Alerting with Dynatrace

Once Oracle Utilities Customer to Meter (C2M) is live, the job is to **see problems before users do** — a slow endpoint, a leaking JVM, a stalled batch worker, an exhausted connection pool. Blind operations finds those the expensive way: a phone call. This solution sets up **Dynatrace monitoring and alerting** for C2M and its **WebLogic 14c** middleware: it instruments the online, batch (OUAF thread pool worker), and integration tiers with the Dynatrace **OneAgent**, builds the service, JVM, and infrastructure **dashboards** that show C2M health at a glance, and defines a **standardized, tuned set of alerts** on the signals that matter — routed to on-call — so the team responds to health, not surprises.

This is a complete, do-it-today package for an experienced C2M / monitoring administrator. Every object is named once — the management zone, the environment tag, the three dashboards, the alerting profile, the five alert rules, the notification — and reused across environments, so you configure it once and repeat it. Deep Dynatrace product specifics (installer flags, metric keys, API shapes) are deferred to the Dynatrace documentation and flagged "verify for your version"; Oracle/MOS-gated content is pointed to, not reproduced.

What you get

- A plain-language Solution Overview of what to monitor across C2M/WebLogic — JVM heap/GC/threads,

WebLogic services and connection pools, batch worker health, and online/IWS endpoints — and how Dynatrace's OneAgent instruments it and its alerting profiles route it.

- A Requirements Guide with verify / implement-if-missing for every prerequisite — a reachable

Dynatrace tenant, installer and read-scoped API tokens, the C2M topology, OneAgent deployability, the JMX/connection-pool metrics, and the alert destination.

- An Implementation Guide of precise, ordered steps: instrument the tiers with OneAgent (host-group

tagged per environment), enable the WebLogic JMX/connection-pool metrics, build the three C2M dashboards, define the standardized alerts as custom metric events, route them with an alerting profile and notification, then pilot, validate coverage, and expand.

- Five automation scripts (shell and the Dynatrace API via curl — not PowerShell) for coverage

discovery, per-tier validation, a coverage/health report, safe remediation (dry-run by default), and a scope/target helper — authentic to C2M / WebLogic administration and Dynatrace read APIs, never printing a token.

- Testing and Troubleshooting guides with explicit pass/fail gates (instrumented, dashboards live,

alerts fire and route), plus Best Practices for alert tuning and noise reduction.

What this delivers

C2M health visible in Dynatrace — tiers instrumented, dashboards live, standardized alerts firing and routed to on-call.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — proven on a non-production pilot before production, with a documented rollback (config export / disable) for every production change.

Outcome

Full observability of C2M: no visibility, user-reported issues, and slow response replaced by instrumented tiers, readable dashboards, and a deliberate, tuned alerting model that the team trusts — validated on a pilot before you scale.

1. Solution Overview — Oracle C2M Monitoring & Alerting with Dynatrace

What this monitors

Once Oracle Utilities Customer to Meter (C2M) is live, the job changes from *installing* it to *seeing it* — catching a slow endpoint, a leaking JVM, or a stalled batch run before a business user does. C2M runs as Java on **Oracle WebLogic Server 14c**: an online tier (the servlet application), one or more **thread pool worker** JVMs that run batch, and integration endpoints (IWS / REST), all over an Oracle database. This solution instruments those tiers with **Dynatrace** (the **OneAgent** deep-code monitor), builds the **service and infrastructure dashboards** that show C2M health at a glance, and defines a standardized set of **alerts** on the signals that actually matter — JVM/heap/GC, WebLogic services and connection pools, batch throughput, and online/IWS endpoints — so the team responds to health, not to surprises. It is configured the repeatable way: named dashboards, named alerting profiles, and named alert rules you define once and reuse across every environment.

> ***Vendor & version boundary.*** *C2M, OUAF, and WebLogic 14c terminology is from Oracle's product and Server Administration documentation; version-specific details (C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented.* ***Dynatrace product internals*** *(OneAgent installer flags, exact Settings paths, metric keys, API request shapes) move between releases: this solution gives the authentic, current approach and* ***defers deep specifics to the Dynatrace documentation*** *rather than reproducing them. Some Oracle notes/patches are gated on My Oracle Support (MOS) and are pointed to, not reproduced. See* `global-requirements-oracle.md`*.*

Why this matters

Blind operations is expensive. Without instrumentation, the first sign of a problem is a phone call: online response has degraded, the nightly batch window overran, an integration is timing out. Without dashboards, "is C2M healthy?" is answered by logging into servers one at a time. Without standardized alerts, either nothing fires (and problems are found late) or everything fires (and the team learns to ignore the noise). A real monitoring solution fixes the cause of blindness: it puts a deep-code agent on every tier so the signals exist, curates them into dashboards a duty operator can read in seconds, and defines a *deliberate, tuned* set of alerts routed to the right destination — turning "did anyone notice?" into "we saw it first."

What Dynatrace gives you on this stack

Dynatrace's **OneAgent** installs once per host and auto-detects the Java processes on it, including WebLogic managed servers and OUAF thread pool workers, providing deep-code tracing, JVM metrics (heap, garbage collection, threads), and service-level timing without per-app configuration. ([Dynatrace Hub — WebLogic](https://www.dynatrace.com/technologies/java-monitoring/weblogic/)) For WebLogic connection-pool depth it exposes the **Java Metric Extensions 2.0 (JMX)** feature / the Connection Pools WebLogic extension. ([Dynatrace Docs — Connection Pools WebLogic](https://docs.dynatrace.com/docs/observe/infrastructure-observability/extensions/connection-pools-weblogic)) Alerting is layered: Dynatrace auto-detects anomalies (for example long garbage-collection time, and out-of-memory / out-of-threads events for Java), and you add **custom metric events** with static thresholds for the signals you care about, then route them with **alerting profiles** filtered by **management zone**, severity, and event type to a notification destination. ([Dynatrace Docs — Metric events](https://www.dynatrace.com/support/help/how-to-use-dynatrace/problem-detection-and-analysis/basic-concepts/event-types/resource-events), [Alerting profiles](https://docs.dynatrace.com/docs/analyze-explore-automate/notifications-and-alerting/alerting-profiles))

The four things to monitor across C2M / WebLogic

1. **JVM health (per tier)** — heap used vs committed, garbage-collection time and suspension, thread

count, and out-of-memory / out-of-threads events. The online tier and each thread pool worker are separate JVMs and are watched separately. Dynatrace alerts by default on long GC time; you add heap-utilization and OOM/OOT alerting explicitly. ([Dynatrace Docs — Java OOM/OOT alerting](https://docs.dynatrace.com/docs/setup-and-configuration/technology-support/application-software/java/set-up-event-and-memory-alerting))

1. **WebLogic services & resources** — managed-server availability, JDBC **connection-pool** capacity

and waits (pool exhaustion stalls the whole application), and stuck/active thread counts on the online servers.

1. **Batch throughput** — the OUAF thread pool workers: are threads completing, is the Batch Job

Submission backlog draining, are runs finishing inside the window. (Batch run status itself lives in the C2M schema and via the JMX batch client; Dynatrace watches the *JVMs and hosts* that run it and can alert on the process health and the database/host signals around it.)

1. **Online & integration endpoints** — response time and failure rate of the key C2M online services

and the IWS/REST inbound integration endpoints, so a slow or failing endpoint is seen as a service problem, not a user complaint.

Named objects this solution defines

So the configuration is repeatable, the artifacts are named once and reused:

• **Management zone** — `C2M - Monitoring Zone`, scoping every C2M host/process so dashboards and

alerting profiles filter to C2M only.

• **Host group tag** — `C2M-Env` (values such as `dev`, `test`, `prod`), applied at OneAgent install so

environment shows on every entity and alerts can be scoped/muted per environment.

- **Dashboards** — `C2M - Service Health` (endpoints, response time, failure rate), `C2M - JVM & Batch`

(heap/GC/threads per tier and batch worker health), and `C2M - Infrastructure` (hosts, CPU/memory, WebLogic connection pools).

- **Alerting profile** — `C2M - Production Alerts`, filtered to the `C2M - Monitoring Zone` and the

`prod` environment, routing to the on-call destination.

- **Alert rules (custom metric events)** — `C2M Alert - JVM Heap High`, `C2M Alert - GC Time High`,

`C2M Alert - Connection Pool Exhausted`, `C2M Alert - Endpoint Failure Rate`, and `C2M Alert - Batch Worker Down`.

- **Notification** — `C2M - OnCall Notification`, the destination (email/Slack/Opsgenie/PagerDuty)

bound to the production alerting profile.

What this is not

This solution instruments C2M and WebLogic with Dynatrace and defines the dashboards and alerts. It does not replace C2M's own batch run-status tables or the JMX batch client (those remain the system of record for batch), does not tune WebLogic or C2M itself (a monitoring signal tells you *where* to tune), and does not reproduce Dynatrace's own product manual — deep OneAgent/API specifics are deferred to the Dynatrace docs. Where the included scripts touch live systems they **read**: discovery, validation, and report are read-only; remediation runs **dry-run by default**. Any change lands on a non-production environment first and goes to production under change control.

The deployment model

> *Assess the C2M topology and what Dynatrace already sees, configure the OneAgent coverage plus the dashboards and standardized alerts, pilot on a non-production environment, validate that coverage is complete and alerts fire correctly against a gate, then expand to production. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*

2. Requirements Guide — Oracle C2M Monitoring & Alerting with Dynatrace

Verify every requirement before instrumenting tiers, building dashboards, or defining alerts. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (licensing, certified stack, environments, WLST/shell/SQL tooling, change control, and the monitoring/support model) are in `global-requirements-oracle.md`. Dynatrace product specifics move between releases — where a step names a Dynatrace path or metric key, **verify it against the current Dynatrace documentation** for your tenant version.

2.1 A Dynatrace tenant reachable from the C2M hosts

Description: A Dynatrace environment (SaaS `https://your-tenant.live.dynatrace.com`, or Managed) that the C2M/WebLogic hosts can reach outbound (directly or via an ActiveGate), with an environment ID you know. Why: OneAgent must reach the tenant to register hosts and ship telemetry, and the dashboards/alerts you build live in that tenant; without it there is nothing to instrument into. Verify: From a C2M host, confirm outbound reachability to the tenant (or the ActiveGate) and note the environment ID; run `OC2MDYNA-Discovery.sh`, which checks tenant reachability (read-only). Implement-if-missing: Provision or obtain access to the tenant (and an ActiveGate if the hosts have no direct egress); confirm the environment ID and base URL.

2.2 A PaaS/installer token and an API token (least privilege)

Description: A **OneAgent installer (PaaS) token** to download/install OneAgent, and a separate **API token** scoped for read (for example metrics read and entities read) used by the discovery and report scripts. Tokens are held in the environment/secrets store — never printed or committed. Why: OneAgent install and the API-based coverage checks each need a token; least-privilege, separate tokens keep install rights and read rights distinct and revocable. Verify: The tokens exist and are stored as environment variables/secrets; run `OC2MDYNA-Discovery.sh` with the API token set (via env var) — it confirms the token authenticates for read (it never echoes the token). Implement-if-missing: Generate the installer token and a read-scoped API token in the Dynatrace tenant (Access tokens), store them in the secrets store, and reference them by env var only.

2.3 The C2M / WebLogic topology in scope

Description: A known inventory of the hosts and JVMs to monitor — the WebLogic AdminServer and managed servers (online tier), the OUAF **thread pool worker** hosts (batch), any IWS/integration nodes, and the database host — with which environment (dev/test/prod) each belongs to. Why: Coverage is only "complete" against a known target list; you cannot confirm every C2M JVM is instrumented if you do not know how many there are. Verify: You can list the C2M hosts/JVMs per environment; run `OC2MDYNA-ScopeHelper.sh --scope pilot` to emit the pilot target list, then `all` for expansion. Implement-if-missing: Compile the topology from the WebLogic domain (managed servers/machines) and the OUAF worker configuration with the infrastructure team; record host, role, and environment for each.

2.4 OneAgent deployable on every C2M host (root/sudo, host group tag)

Description: The ability to install OneAgent on each in-scope host (root/sudo for the install), and a decided **host group** naming so environment is tagged at install time — this solution uses the host group tag `C2M-Env` with a value per environment (`dev`/`test`/`prod`). Why: Deep-code monitoring of the WebLogic and worker JVMs comes from OneAgent on the host; the host group/tag is what lets dashboards and alerting profiles scope and mute per environment. Verify: You can run the installer on a host and set its host group; run `OC2MDYNA-Validation.sh`, which reports which in-scope hosts already report a running OneAgent and whether the `C2M-Env` tag is set. Implement-if-missing: Obtain install privileges and agree the host-group value per environment before the pilot install.

2.5 WebLogic connection-pool / JMX metrics enabled

Description: The Dynatrace **Java Metric Extensions 2.0 (JMX)** feature (or the Connection Pools WebLogic extension) enabled so JDBC connection-pool depth, waits, and capacity are captured for the managed servers. Why: Connection-pool exhaustion stalls the whole online application; without the

JMX/extension metrics there is no pool-depth signal to dashboard or alert on. ([Dynatrace Docs — Connection Pools WebLogic](https://docs.dynatrace.com/docs/observe/infrastructure-observability/extensions/connection-pools-weblogic)) Verify: Pool metrics appear for the WebLogic processes in Dynatrace; `OC2MDYNA-Validation.sh` checks whether connection-pool metrics are present for the in-scope managed servers. Implement-if-missing: Enable the JMX metric extension / Connection Pools WebLogic extension per the current Dynatrace documentation and confirm the metrics populate.

2.6 Environment access to build dashboards, alerts, and notifications

Description: Dynatrace user access with permission to create **management zones**, **dashboards**, **metric events (custom alerts)**, **alerting profiles**, and **notification integrations**, plus the details of the notification destination (email list, or the Slack/Opsgenie/PagerDuty target). Why: The deliverable is the configured dashboards and the standardized, routed alerts; without config rights and a destination there is nowhere for an alert to go. Verify: You can create a test dashboard and a test alerting profile in the tenant, and you have the destination details; recorded against the checklist. Implement-if-missing: Obtain the tenant role/permissions and confirm the on-call notification destination with the operations team before building alerts.

2.7 Alert destination and on-call routing decided

Description: A decided routing for C2M production alerts — who is notified, by what channel, and the severity that pages versus emails — realized as the notification `C2M - OnCall Notification` bound to the `C2M - Production Alerts` profile. Why: An alert nobody receives is not an alert; standardized routing is what turns a fired problem into a response, and per-environment scoping keeps non-production noise off the on-call channel. Verify: The destination is confirmed and a test problem/notification reaches it; validated in the pilot. Implement-if-missing: Agree the channel and on-call schedule with operations; create the notification integration and bind it to the production alerting profile.

2.8 A non-production pilot environment and change control

Description: A non-production C2M environment to instrument, dashboard, and prove the alerts on before production, plus change control and backups (config exports of dashboards/alerting profiles) per the global requirements. Why: Instrumentation and alert definitions are changes to a live monitoring surface; the model is Assess → Configure → Pilot → Validate → Expand with a documented rollback (delete/disable the added alert or restore the prior dashboard export). Verify: A non-production environment is available and you can export/re-import the dashboard and alerting-profile config for rollback. Implement-if-missing: Stand up or obtain the non-production environment and establish the config-export backup and change window before any production change.

Checklist

- Dynatrace tenant reachable from C2M hosts, environment ID known — `OC2MDYNA-Discovery.sh`
- Installer (PaaS) token + read-scoped API token, stored as secrets (never printed) — `OC2MDYNA-Discovery.sh`
- C2M/WebLogic topology in scope (online, workers, IWS, DB) per environment — `OC2MDYNA-ScopeHelper.sh`

• OneAgent deployable on every host; host-group tag `C2M-Env` decided — `OC2MDYNA-Validation.sh`

• WebLogic connection-pool / JMX metrics enabled — `OC2MDYNA-Validation.sh`

• Access to build management zone, dashboards, metric events, alerting profiles, notifications

• Alert destination + on-call routing decided — `C2M - OnCall Notification`

• Non-production pilot + change control and config-export backups

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS

accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration

Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic,

the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only

workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL

connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain

config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a

documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and

release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M Monitoring & Alerting with Dynatrace

Follow these steps in order. This guide instruments the C2M / WebLogic 14c tiers with Dynatrace OneAgent first, then builds the C2M dashboards, then defines the standardized alerts and their notification, then validates coverage — pilot-first on a non-production environment, then expand to production under change control. Object names are fixed — create them with the exact names shown and reuse them everywhere.

> *Model: assess what Dynatrace already sees, configure coverage plus dashboards and alerts, prove it on a non-production pilot, validate against the gate, then expand to production. Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*
>
> ***Dynatrace specifics move between releases.** Installer flags, Settings menu paths, metric keys, and API request shapes below are the current, authentic approach; **verify each against the Dynatrace documentation for your tenant version** before running it. Never print or commit a token — reference it from an environment variable / secrets store.*

Step 1 — Assess current coverage

1. Run `OC2MDYNA-ScopeHelper.sh --scope pilot` to fix the pilot target list (the WebLogic online servers, the OUAF thread pool worker hosts, IWS nodes, and the DB host for the pilot environment), then `--scope all` for the full expansion set.

1. With the read-scoped API token set as an environment variable, run `OC2MDYNA-Discovery.sh`. It confirms tenant reachability and reports which in-scope hosts already report a running OneAgent and which C2M JVMs/processes Dynatrace already sees (read-only). Record gaps against the Requirements Guide checklist. Do not proceed past a missing tenant, missing token, or unknown topology — implement those first.

Step 2 — Instrument the tiers with OneAgent

1. In the Dynatrace tenant, obtain the OneAgent Linux install command for your environment (it embeds the environment URL and the installer token). ([Dynatrace Docs — Install OneAgent on Linux](https://docs.dynatrace.com/docs/ingest-from/dynatrace-oneagent/installation-and-operation/linux/installation/install-oneagent-on-linux))

1. On each in-scope host, install OneAgent as root/sudo and set the **host group** so the environment

is tagged from the start — this solution standardizes the host-group tag `C2M-Env` with a value per environment:

```
# download (token supplied via env var — never inline it), then install with the host group set
sudo /bin/sh Dynatrace-OneAgent-Linux.sh --set-host-group=C2M-Env-prod
```

(The exact installer name and the `--set-host-group` flag are Dynatrace's — verify for your version. Installing OneAgent is sufficient to deep-monitor the WebLogic managed servers and the OUAF thread pool worker JVMs on that host; no per-app configuration is required. ([Dynatrace Hub — WebLogic](https://www.dynatrace.com/technologies/java-monitoring/weblogic/)))

1. Enable the **Java Metric Extensions 2.0 (JMX)** feature / the Connection Pools WebLogic extension so

JDBC connection-pool depth and waits are captured for the managed servers (Requirement 2.5). ([Dynatrace Docs — Connection Pools WebLogic](https://docs.dynatrace.com/docs/observe/infrastructure-observability/extensions/connection-pools-weblogic))

1. Run `OC2MDYNA-Validation.sh` to confirm each pilot host now reports a running OneAgent with the

`C2M-Env` tag and that connection-pool metrics are present.

Step 3 — Define the management zone and environment tag

1. Create the management zone `C2M - Monitoring Zone` scoped to the C2M hosts/processes (by the host

group / tag). This is what every dashboard and alerting profile filters on so they see C2M only.

1. Confirm the `C2M-Env` host-group tag is present on every entity (it flows from the install in Step 2)

so alerts can be scoped and muted per environment.

Step 4 — Build the C2M dashboards

Build three named dashboards so C2M health reads at a glance. (Use the tenant's dashboard editor; tile/metric names are the current Dynatrace ones — verify for your version.)

1. `C2M - Service Health` — the online and IWS/REST endpoints: response time, failure rate, and

throughput of the key C2M services, plus service availability. This is the "are users being served" view.

1. `C2M - JVM & Batch` — per tier (online servers and each thread pool worker): heap used vs

committed, garbage-collection time / suspension, thread count, and worker process availability. This is the "are the JVMs healthy and is batch running" view.

1. `C2M - Infrastructure` — hosts (CPU, memory, disk), WebLogic managed-server availability, and the

JDBC **connection-pool** depth/waits from the JMX extension. This is the "is the platform under it healthy" view. Scope all three to the `C2M - Monitoring Zone`.

Step 5 — Define the standardized alerts (custom metric events)

Dynatrace already auto-detects some conditions (long garbage-collection time; out-of-memory / out-of-threads events for Java). ([Dynatrace Docs — Java OOM/OOT alerting](https://docs.dynatrace.com/docs/setup-and-configuration/technology-support/application-software/java/set-up-event-and-memory-alerting)) Add the following **custom metric events** with static thresholds (Settings → Anomaly detection → Metric events, or the equivalent for your version) so the signals that matter to C2M alert deliberately. Name each exactly:

- `C2M Alert - JVM Heap High` — heap-utilization percentage over threshold, sustained (for example

used/committed high for several intervals), per WebLogic/worker JVM.

- `C2M Alert - GC Time High` — garbage-collection time / suspension above threshold (complements

Dynatrace's built-in long-GC detection with your own tuned bound).

- `C2M Alert - Connection Pool Exhausted` — JDBC active-vs-max connections at capacity or waiters

present, from the JMX/Connection Pools extension.

- `C2M Alert - Endpoint Failure Rate` — failure rate of a key online/IWS service over threshold.

- `C2M Alert - Batch Worker Down` — a thread pool worker process/host not reporting (worker JVM

unavailable). Give each a clear title, threshold, and (where useful) a dimension split per entity, and keep them in the `C2M - Monitoring Zone`. Static thresholds are the metric-key-based event type; use auto-adaptive or seasonal baselines only where a fixed number does not fit. ([Dynatrace Docs — Metric events](https://www.dynatrace.com/support/help/how-to-use-dynatrace/problem-detection-and-analysis/basic-concepts/event-types/resource-events))

Step 6 — Route alerts with an alerting profile and notification

1. Create the alerting profile `C2M - Production Alerts`. Set its scope to the `C2M - Monitoring Zone`

and filter to the `prod` value of `C2M-Env`, with severity rules and event filters for the C2M alert rules from Step 5. An event must satisfy the profile's management-zone, severity, and event conditions (AND) to notify. ([Dynatrace Docs — Alerting profiles](https://docs.dynatrace.com/docs/analyze-explore-automate/notifications-and-alerting/alerting-profiles))

1. Create the notification `C2M - OnCall Notification` (email, or a Slack/Opsgenie/PagerDuty

integration) and bind it to the `C2M - Production Alerts` profile so C2M production problems reach on-call. Non-production stays off this profile so it does not page.

Step 7 — Pilot on non-production

1. On the non-production environment, confirm every pilot tier is instrumented, the three dashboards

render real data, and the alert rules exist and are scoped to the zone.

1. Run `OC2MDYNA-Validation.sh` to confirm OneAgent coverage across the pilot topology, the `C2M-Env`

tag, connection-pool metrics, and that the named dashboards/alerting profile exist.

1. Deliberately exercise an alert in non-production (for example drive heap or endpoint failure on a

throwaway load) and confirm the metric event fires and — via a non-production test profile — routes as expected. Capture the state with `OC2MDYNA-Report.sh`.

Step 8 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when: every in-scope tier reports a running

OneAgent with the `C2M-Env` tag; the three dashboards show live C2M data; connection-pool metrics are present; every named alert rule exists in the zone; and at least one alert was proven to fire and route.

1. Remediate any coverage or config gap with `OC2MDYNA-Remediation.sh` (dry-run by default — it prints

the intended action, such as the OneAgent install command for a missing host or the missing tag/alert to create; apply only in a change window) before re-running validation.

Step 9 — Expand to production

1. After the pilot passes the gate, instrument the remaining production hosts (`--set-host-group=

C2M-Env-prod`), and create the same management zone, dashboards, alert rules, alerting profile, and notification in production, with a config export of the dashboards/alerting profiles taken first as the rollback.

1. Re-run `OC2MDYNA-Validation.sh` and `OC2MDYNA-Report.sh` in production; keep the dashboards as the

standing operational view and confirm the on-call notification is live so C2M problems are seen first, not reported by users.

Order of operations

```
Assess coverage -> Instrument tiers (OneAgent + JMX) -> Management zone & tag -> Build dashboards -> Define standardized alerts -> Route via profile + notification -> Pilot -> Validate gate -> Expand to production
```

5. Testing Guide — Oracle C2M Monitoring & Alerting with Dynatrace

Run after the pilot on the non-production environment (Implementation Step 7) and again after production expansion. Each test confirms one part of the monitoring and alerting is working end to end. Set the read-scoped API token as an environment variable first (never inline it); the scripts never print it.

Test 1 — Every in-scope tier is instrumented

Do: Run `OC2MDYNA-Validation.sh` against the scope from `OC2MDYNA-ScopeHelper.sh`. Expected: Every in-scope host (WebLogic online servers, each OUAF thread pool worker, IWS nodes, DB host) reports a **running OneAgent**, and Dynatrace sees the WebLogic and worker JVMs — no in-scope host missing coverage.

Test 2 — The environment tag is set

Do: Check the host group / tag on the instrumented hosts; `OC2MDYNA-Validation.sh` reports it. Expected: Every C2M host carries the `C2M-Env` tag with the correct environment value (`dev`/`test`/ `prod`) so dashboards and alerting profiles can scope and mute per environment.

Test 3 — Connection-pool metrics are present

Do: Confirm JDBC connection-pool metrics populate for the managed servers; `OC2MDYNA-Validation.sh` checks it. Expected: Pool depth/waits appear for the in-scope WebLogic managed servers (the JMX / Connection Pools extension is active) — pool exhaustion is observable.

Test 4 — The dashboards render live C2M data

Do: Open `C2M - Service Health`, `C2M - JVM & Batch`, and `C2M - Infrastructure`. Expected: Each dashboard shows real, current data scoped to the `C2M - Monitoring Zone` — endpoints and failure rate, per-tier heap/GC/threads and worker health, and hosts plus connection pools — with no empty tiles for an in-scope tier.

Test 5 — The standardized alert rules exist and are scoped

Do: List the metric events / alert rules; `OC2MDYNA-Validation.sh` reports their presence. Expected: All five named rules exist in the zone — `C2M Alert - JVM Heap High`, `C2M Alert - GC Time High`, `C2M Alert - Connection Pool Exhausted`, `C2M Alert - Endpoint Failure Rate`, `C2M Alert - Batch Worker Down` — each with a threshold.

Test 6 — An alert fires on a real condition (non-production)

Do: On a throwaway non-production load, drive a monitored signal past its threshold (for example heap utilization or endpoint failure rate). Expected: The corresponding metric event opens a problem in Dynatrace within its detection window — the alert actually fires on the condition it is meant to catch.

Test 7 — The alert routes to the destination

Do: With a non-production test alerting profile bound to the destination, confirm the fired problem notifies. Expected: The notification reaches `C2M - OnCall Notification` (or its non-production test equivalent) — the on-call destination receives the alert. Production `C2M - Production Alerts` is scoped to `prod` only, so non-production does not page the on-call channel.

Test 8 — Coverage report is clean

Do: Run `OC2MDYNA-Report.sh` and review the coverage/health snapshot. Expected: The report shows all in-scope tiers instrumented and tagged, dashboards and alert rules present, and no coverage gaps —

the artifact you attach to the change record and reuse as a baseline.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass on non-production: every tier instrumented and tagged, connection-pool metrics present, dashboards live, all five alert rules exist, and an alert proven to fire; plus Test 7 (routing) verified with a non-production test profile
Expand pass	Tests 1-5 and 8 pass in production after instrumenting the production tiers, and the `C2M - Production Alerts` profile + `C2M - OnCall Notification` are live and confirmed reachable

6. Troubleshooting Guide — Oracle C2M Monitoring & Alerting with Dynatrace

Use `OC2MDYNA-Discovery.sh` and `OC2MDYNA-Validation.sh` for coverage and configuration state, `OC2MDYNA-Report.sh` for the coverage/health snapshot, and `OC2MDYNA-Remediation.sh` (dry-run by default) for safe fixes to coverage/config gaps. Set the read-scoped API token as an environment variable first — the scripts never print it. Dynatrace product specifics move between releases: verify paths, metric keys, and API shapes against the current Dynatrace documentation for your tenant version. Oracle/MOS-gated notes are pointed to, not reproduced.

A. A host is not reporting an OneAgent

Symptom	Cause	Remediation
Host absent from coverage	OneAgent not installed on that host	Install OneAgent (`--set-host-group=C2M-Env-<env>`); `OC2MDYNA-Remediation.sh` prints the install command (dry-run)
Host present but stale/offline	Agent stopped, or host lost egress to the tenant	Confirm the OneAgent service is running and the host can reach the tenant/ActiveGate; re-check with `OC2MDYNA-Validation.sh`
JVMs not seen though host is up	Deep monitoring not active for that process	Confirm OneAgent deep monitoring is enabled for the host and the WebLogic/worker processes are running; verify Java support for your OS per Dynatrace docs

B. A C2M JVM is instrumented but a metric is missing

Symptom	Cause	Remediation
No connection-pool metrics	JMX / Connection Pools WebLogic extension not enabled	Enable the Java Metric Extensions 2.0 (JMX) feature / Connection Pools extension per Dynatrace docs; `OC2MDYNA-Validation.sh` re-checks presence
No batch worker signal	Worker process not detected / worker host not covered	Confirm the thread pool worker host has OneAgent and the worker JVM is running; scope it in `OC2MDYNA-ScopeHelper.sh`
Endpoint timing absent	Service not detected / no traffic during the window	Generate representative traffic; confirm the online/IWS service is detected before dashboarding it

C. The environment tag is wrong or missing

- An entity has no `C2M-Env` tag, or the wrong value, so it slips a dashboard filter or an alerting

profile scope. The host group was not set (or set wrong) at install. Re-install/adjust the host group to the correct `C2M-Env-<env>`; `OC2MDYNA-Validation.sh` reports which hosts lack the tag. Tag at install time — retro-tagging a live fleet is error-prone.

D. A dashboard tile is empty

Symptom	Cause	Remediation
Tile empty for an in-scope tier	That tier not instrumented, or metric not populated	Confirm coverage (Section A) and the underlying metric (Section B) before assuming the tile is wrong
Whole dashboard empty	Management-zone scope excludes the entities	Confirm `C2M - Monitoring Zone` includes the C2M hosts/processes (by host group/tag)

E. An alert does not fire

Symptom	Cause	Remediation
Condition clearly breached, no problem	Metric event threshold/window too loose, or wrong metric selected	Re-check the metric event's metric, threshold, and detection window; tune off the pilot baseline (Best Practices)

Symptom	Cause	Remediation
Rule missing entirely	Alert rule never created in the zone	`OC2MDYNA-Validation.sh` reports which of the five named rules are missing; create them per Implementation Step 5
Fires but only sometimes	Sustained-breach requirement not met	Confirm the "for N intervals" condition matches how the signal behaves; C2M batch spikes may need a sustained window, not a single sample

F. An alert fires but nobody is notified

- A problem opens in Dynatrace but on-call gets nothing. The alerting profile or notification is

misconfigured. Confirm `C2M - Production Alerts` scopes the `C2M - Monitoring Zone` and matches the event/severity, and that `C2M - OnCall Notification` is bound to it and points at the live destination (email/Slack/Opsgenie/PagerDuty). Profile conditions combine with AND — an over-tight filter silences everything. ([Dynatrace Docs — Alerting profiles](https://docs.dynatrace.com/docs/analyze-explore-automate/notifications-and-alerting/alerting-profiles))

G. Too many alerts / alert fatigue

- The on-call channel is noisy. Either a threshold is set inside the normal range, or non-production is

leaking onto the production profile. Re-tune the threshold above the observed healthy baseline, and confirm `C2M - Production Alerts` is filtered to the `prod` value of `C2M-Env` so non-production never pages. Use maintenance windows / muting for planned batch and patching rather than deleting rules.

H. Diagnostics quick reference

```
export DT_API_TOKEN=...            # read-scoped token, from the secrets store (never committed/echoed)
export DT_ENV_URL=https://your-tenant.live.dynatrace.com
./OC2MDYNA-Discovery.sh            # tenant reachable; which hosts/JVMs are already seen (read-only)
./OC2MDYNA-Validation.sh           # coverage, C2M-Env tag, pool metrics, dashboards/alerts present (read-
./OC2MDYNA-Report.sh               # one coverage/health snapshot to attach to a change record (read-only)
./OC2MDYNA-Remediation.sh          # dry-run by default; prints the intended fix (install/tag/alert)
```

I. When to escalate

If every in-scope tier reports a running OneAgent, the dashboards render, the alert rules exist and are scoped, and an expected condition still does not alert or route after tuning, capture `OC2MDYNA-Validation.sh` and `OC2MDYNA-Report.sh` output and review the metric-event and alerting-profile configuration against the current Dynatrace documentation for your tenant version — the mechanics (metric keys, profile filters) may have changed release-to-release. For C2M/WebLogic-side questions consult the version's My Oracle Support notes and open a service request — gated content is pointed to, not reproduced here.

7. Best Practices — Oracle C2M Monitoring & Alerting with Dynatrace

• Alert on the signal, not on everything. Ship the deliberate, standardized set — heap, GC, connection pool, endpoint failure rate, batch worker down — and resist adding an alert for every metric on the dashboard. A curated set that always means "act now" beats a hundred rules the team learns to ignore.

• Tune thresholds off the pilot's own baseline, not a guess. Watch each signal on `C2M - JVM & Batch` and `C2M - Service Health` over a normal cycle, then set the static threshold above the observed healthy range so the alert marks a real deviation. Re-tune after the first weeks in production; a threshold that fires nightly on normal batch is noise, not signal.

• Let Dynatrace's built-in detection do its job, then add yours. Dynatrace already alerts on long garbage-collection time and on out-of-memory / out-of-threads events for Java; add `C2M Alert - GC Time High` and `C2M Alert - JVM Heap High` as your own tuned bounds rather than duplicating what the platform already catches. ([Dynatrace Docs — Java OOM/OOT alerting](https://docs.dynatrace.com/docs/setup-and-configuration/technology-support/application-software/java/set-up-event-and-memory-alerting))

• Scope every alerting profile to the `C2M - Monitoring Zone` and the environment. Filter `C2M - Production Alerts` to `prod` so non-production never pages on-call; management zone + severity + event filters combine with AND, so a tight scope is a quiet on-call channel. ([Dynatrace Docs — Alerting profiles](https://docs.dynatrace.com/docs/analyze-explore-automate/notifications-and-alerting/alerting-profiles))

• Tag environment at install time, not after. Set the `C2M-Env` host group when you install OneAgent so every entity carries it from the first data point — retro-tagging a live fleet is error-prone, and every dashboard filter and alert scope depends on it.

• One OneAgent per host covers all the JVMs on it. You do not instrument WebLogic and each thread pool worker separately — installing OneAgent on the host deep-monitors every Java process on it. Confirm coverage per *host*, and confirm the JVMs are *seen* per tier. ([Dynatrace Hub — WebLogic](https://www.dynatrace.com/technologies/java-monitoring/weblogic/))

• Watch the connection pool as closely as the JVM. Pool exhaustion stalls the whole online application while the JVM still looks "up"; the JMX / Connection Pools extension is what makes it visible, so keep that metric on `C2M - Infrastructure` and alerted via `C2M Alert - Connection Pool Exhausted`.

• Keep the dashboards role-shaped and stable. `C2M - Service Health` for the duty operator, `C2M - JVM & Batch` for the middleware/app team, `C2M - Infrastructure` for the platform team — name them once and reuse across environments so everyone reads the same view.

• Mute deliberately during maintenance, do not delete. Use maintenance windows / profile muting for planned batch or patching so expected conditions do not page, rather than deleting the alert and forgetting to restore it.

• Version-control your monitoring config. Export dashboards and alerting profiles (config as JSON) so a change is reviewable and reversible — that export is your rollback when you expand to production.

• Prove the alert path before you rely on it. An alert that has never fired is untested; exercise each new rule on a throwaway non-production load and confirm it reaches `C2M - OnCall Notification` before trusting it in production.

• Defer deep Dynatrace specifics to the vendor docs. Metric keys, Settings paths, and API shapes change between Dynatrace releases; keep this solution's names and thresholds, but verify the mechanics against the current Dynatrace documentation for your tenant version.

8. Deliverables Summary — Oracle C2M Monitoring & Alerting with Dynatrace

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What to monitor across C2M/WebLogic (JVM, services, batch, endpoints) and how Dynatrace instruments and alerts on it (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: instrument tiers with OneAgent + JMX, build the C2M dashboards, define standardized alerts + notification, validate coverage
5. Testing Guide	Validation procedures with pass/fail gates (instrumented, dashboards live, alerts fire and route)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for alert tuning and noise reduction

Named objects

Object	Name
Management zone	`C2M - Monitoring Zone`
Environment host-group tag	`C2M-Env` (`dev` / `test` / `prod`)
Service dashboard	`C2M - Service Health`
JVM/batch dashboard	`C2M - JVM & Batch`

Object	Name
Infrastructure dashboard	`C2M - Infrastructure`
Production alerting profile	`C2M - Production Alerts`
Alert — JVM heap	`C2M Alert - JVM Heap High`
Alert — GC time	`C2M Alert - GC Time High`
Alert — connection pool	`C2M Alert - Connection Pool Exhausted`
Alert — endpoint failures	`C2M Alert - Endpoint Failure Rate`
Alert — batch worker	`C2M Alert - Batch Worker Down`
Notification	`C2M - OnCall Notification`

Automation scripts (shell & Dynatrace API via curl — not PowerShell)

Script	What it does
`OC2MDYNA-Discovery.sh`	Confirms tenant reachability and reports which in-scope hosts report a running OneAgent and which C2M JVMs Dynatrace already sees (read-only; never prints the token)
`OC2MDYNA-Validation.sh`	Confirms every in-scope tier is instrumented and `C2M-Env`-tagged, connection-pool metrics are present, and the named dashboards/alert rules exist (read-only)
`OC2MDYNA-Report.sh`	Renders one monitoring coverage/health snapshot (tiers instrumented, tags, dashboards, alert rules, gaps) to attach to a change record and reuse as a baseline (read-only)
`OC2MDYNA-Remediation.sh`	Prints the intended fix for a coverage/config gap (OneAgent install command, missing tag, missing alert); dry-run by default, applies only with an explicit flag in a change window
`OC2MDYNA-ScopeHelper.sh`	Builds the in-scope C2M host / tier target list for pilot and expansion from a scope file

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five automation scripts (shell / Dynatrace API via curl) in a single ZIP.
- A repeatable configuration following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

C2M health visible in Dynatrace — the WebLogic and thread pool worker tiers instrumented, the service / JVM / infrastructure dashboards live, and a standardized, tuned set of alerts firing and routed to on-call — so the team responds to problems before users feel them, proven on a non-production pilot before it is expanded to production under change control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MDYNA-Discovery.sh

Purpose: Capture current-state monitoring coverage before you instrument or dashboard C2M. Confirms the Dynatrace tenant is reachable, then reports which hosts already run a OneAgent and which C2M/WebLogic JVMs and processes Dynatrace already sees. Uses the Dynatrace read APIs with a read-scoped token supplied via env var. Read-only — it only queries the tenant and changes nothing. It never prints the token. Tenant placeholders use plain text (your-tenant, your-token) without angle brackets. HOW TO RUN Set the tenant URL and a read-scoped API token as environment variables first, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Discovery.sh Optional: HOST_FILTER=C2M-Env to narrow entity queries to the C2M host group/tag. EXPECTED RESULTS Prints: tenant reachability PASS/FAIL, the count of hosts reporting a running OneAgent, and a list of monitored hosts and detected process groups (WebLogic / thread pool worker JVMs) it can see. PASS = tenant reachable and at least the expected C2M hosts appear. FAIL = tenant unreachable or token invalid — fix first. NOTES White-label, repeatable coverage discovery. Read-only where it reads live systems: it only issues GET requests to the Dynatrace API. The token is read from an env var and never echoed. Dynatrace API paths/fields move between releases — verify against the current Dynatrace documentation for your tenant version.

How to run: Set the tenant URL and a read-scoped API token as environment variables first, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Discovery.sh Optional: HOST_FILTER=C2M-Env to narrow entity queries to the C2M host group/tag. EXPECTED RESULTS Prints: tenant reachability PASS/FAIL, the count of hosts reporting a running OneAgent, and a list of monitored hosts and detected process groups (WebLogic / thread pool worker JVMs) it can see. PASS = tenant reachable and at least the expected C2M hosts appear. FAIL = tenant unreachable or token invalid — fix first. NOTES White-label, repeatable coverage discovery. Read-only where it reads live systems: it only issues GET requests to the Dynatrace API. The token is read from an env var and never echoed. Dynatrace API paths/fields move between releases — verify against the current Dynatrace documentation for your tenant version.

OC2MDYNA-Remediation.sh

Purpose: When validation flags a gap — a host without OneAgent, a missing C2M-Env tag, or a missing named alert rule — print the exact, safe remediation the admin should apply: the OneAgent install command, the host-group tag to set, or the alert rule to create. DRY-RUN by default: it prints what it WOULD do and changes nothing. It applies only when explicitly asked, and only for the tag/alert config it is authorized to write via the Dynatrace API — it never installs software itself and never prints the API token. Placeholders are plain text (your-tenant, your-token) without angle brackets. HOW TO RUN Set the tenant URL and (for --apply) a suitably scoped API token as env vars, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-scoped-token ./OC2MDYNA-Remediation.sh # DRY-RUN: prints intended fixes only ./OC2MDYNA-Remediation.sh --apply # applies config it is authorized to (change window) Override env value: ENV_VALUE=prod ./OC2MDYNA-Remediation.sh EXPECTED RESULTS DRY-RUN prints, for each gap type, the exact action: the OneAgent install command (host-group tagged), the tag to set, and the alert-rule names to

create — and exits without changing anything. With --apply it performs only the config writes it is explicitly authorized for (never software install) and prints the manual steps for the rest. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never installs OneAgent itself (that is a root/sudo host action for the admin) and never restarts a service. The token is read from an env var and never echoed. Dynatrace install flags / API shapes move between releases — verify for your tenant version.

How to run: Set the tenant URL and (for --apply) a suitably scoped API token as env vars, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-scoped-token ./OC2MDYNA-Remediation.sh # DRY-RUN: prints intended fixes only ./OC2MDYNA-Remediation.sh --apply # applies config it is authorized to (change window) Override env value: ENV_VALUE=prod ./OC2MDYNA-Remediation.sh EXPECTED RESULTS DRY-RUN prints, for each gap type, the exact action: the OneAgent install command (host-group tagged), the tag to set, and the alert-rule names to create — and exits without changing anything. With --apply it performs only the config writes it is explicitly authorized for (never software install) and prints the manual steps for the rest. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never installs OneAgent itself (that is a root/sudo host action for the admin) and never restarts a service. The token is read from an env var and never echoed. Dynatrace install flags / API shapes move between releases — verify for your tenant version.

OC2MDYNA-Report.sh

Purpose: Produce one coverage and health snapshot of C2M monitoring in Dynatrace — hosts monitored and C2M-Env-tagged, C2M/WebLogic JVMs seen, whether connection-pool metrics exist, and which named dashboards and alert rules are present — to attach to a change record and reuse as a regression baseline. Reads only; changes nothing. It never prints the API token. Placeholders are plain text (your-tenant, your-token). HOW TO RUN Set the tenant URL and a read-scoped API token as environment variables, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Report.sh # prints to screen ./OC2MDYNA-Report.sh --out oc2mdyna-report.txt # also writes a text file EXPECTED RESULTS A sectioned report (Tenant, Hosts & tags, Detected JVMs, Metrics, Dashboards, Alert rules, Findings) to screen and, if --out is given, to a file. A clean estate shows all hosts monitored and tagged, pool metrics present, all three dashboards and all five alert rules present, and zero findings. Exit 0 on success. NOTES White-label, repeatable: capture a report before and after every change so a green-to-green comparison proves you did not regress coverage. Strictly read-only (GET requests only); the token is read from an env var and never echoed. Dynatrace API paths/fields move between releases — verify for your tenant version.

How to run: Set the tenant URL and a read-scoped API token as environment variables, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Report.sh # prints to screen ./OC2MDYNA-Report.sh --out oc2mdyna-report.txt # also writes a text file EXPECTED RESULTS A sectioned report (Tenant, Hosts & tags, Detected JVMs, Metrics, Dashboards, Alert rules, Findings) to screen and, if --out is given, to a file. A clean estate shows all hosts monitored and tagged, pool metrics present, all three dashboards and all five alert rules present, and zero findings. Exit 0 on success. NOTES White-label, repeatable: capture a report before and after every change so a green-to-green comparison proves you did not regress coverage. Strictly read-only (GET requests only); the token is read from an env var and never echoed. Dynatrace API paths/fields move between releases — verify for your tenant version.

OC2MDYNA-ScopeHelper.sh

Purpose: Produce a bounded, reusable scope for the monitoring work: the C2M hosts and their tier (online WebLogic, thread pool worker, IWS, database) and environment, so the pilot and each expansion wave run against a known set rather than "everything at once". It reads a scope definition file and emits a clean, de-duplicated target list the discovery, validation, and report scripts consume. Read-only — it only reads the scope file. Placeholders are plain text (your-host) without angle brackets. HOW TO RUN No tenant token needed — this only reads a local scope file: ./OC2MDYNA-ScopeHelper.sh --scope pilot # hosts/tiers tagged pilot ./OC2MDYNA-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mdyna-scope.conf) lines: HOST,TIER,ENV,SCOPE e.g. your-wls-01,online,prod,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: hosts with their tier and environment, and a per-tier count. Use the output to drive OneAgent install (Implementation Step 2) and to target discovery/validation/report runs. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each expansion wave, so scope is explicit and auditable. Read-only. No live-system changes and no token use. Host names and tiers are yours to fill from the WebLogic domain / OUAF worker topology (and cross-checked against OC2MDYNA-Discovery.sh output).

How to run: No tenant token needed — this only reads a local scope file: ./OC2MDYNA-ScopeHelper.sh --scope pilot # hosts/tiers tagged pilot ./OC2MDYNA-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mdyna-scope.conf) lines: HOST,TIER,ENV,SCOPE e.g. your-wls-01,online,prod,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: hosts with their tier and environment, and a per-tier count. Use the output to drive OneAgent install (Implementation Step 2) and to target discovery/validation/report runs. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each expansion wave, so scope is explicit and auditable. Read-only. No live-system changes and no token use. Host names and tiers are yours to fill from the WebLogic domain / OUAF worker topology (and cross-checked against OC2MDYNA-Discovery.sh output).

OC2MDYNA-Validation.sh

Purpose: Confirm the configured monitoring matches the standard: every in-scope C2M host reports a running OneAgent and carries the C2M-Env tag, WebLogic connection-pool metrics are present, and the named dashboards and alert rules exist in the tenant. Read-only — it validates state and reports PASS/FAIL, it changes nothing. Run it on the pilot and after each expansion. It never prints the API token. Placeholders are plain text (your-tenant, your-token) with no angle brackets. HOW TO RUN Set the tenant URL and a read-scoped API token as environment variables, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Validation.sh --scope pilot # uses oc2mdyna-scope.conf hosts Optional: ENV_TAG=C2M-Env to name the environment host-group tag to check. EXPECTED RESULTS For each check prints PASS or FAIL: every in-scope host has a running OneAgent and the C2M-Env tag; connection-pool metrics are present; each named dashboard and each of the five named alert rules exists. Overall PASS = full coverage, tags, pool metrics, dashboards, and alerts present. Any FAIL names the gap to remediate. NOTES White-label, repeatable validation — the same checks every tier, every environment. Read-only (GET requests only); the token is read from an env var and never echoed. Dashboard/alert names must match the solution's fixed names. Dynatrace API paths and metric keys move between releases — verify for your tenant version.

How to run: Set the tenant URL and a read-scoped API token as environment variables, then: export DT_ENV_URL=https://your-tenant.live.dynatrace.com export DT_API_TOKEN=your-read-scoped-token ./OC2MDYNA-Validation.sh --scope pilot # uses oc2mdyna-scope.conf hosts Optional: ENV_TAG=C2M-Env to name the environment host-group tag to check. EXPECTED RESULTS For each check prints PASS or FAIL: every in-scope host has a running OneAgent and the C2M-Env tag; connection-pool metrics are present; each named dashboard and each of the five named alert rules exists. Overall PASS = full coverage, tags, pool metrics, dashboards, and alerts present. Any FAIL names the gap to remediate. NOTES White-label, repeatable validation — the same checks every tier, every environment. Read-only (GET requests only); the token is read from an env var and never echoed. Dashboard/alert names must match the solution's fixed names. Dynatrace API paths and metric keys move between releases — verify for your tenant version.